Low-Level Design (LLD)

1. Purpose

This document describes the current implementation-level structure of the tailoring marketplace: frontend routing and API access, backend modules, endpoint groups, persisted entities, and key request flows.

2. Frontend Structure

Area	Location	Responsibilities
Application routing	`frontend/src/App.tsx`	Public routes and role-protected customer, tailor, and admin dashboard routes.
Authentication state	`frontend/src/hooks/useAuth.tsx`	Restore the saved session, verify the current user, and expose login/register/logout actions.
Route authorization	`frontend/src/components/common/ProtectedRoute.tsx`	Show loading state, redirect anonymous users, and prevent access to dashboards for the wrong role.
API transport	`frontend/src/services/api.ts`	Build requests from the configured API base URL, attach the bearer token, apply a 10-second timeout, parse errors, and clear stale session data on 401 responses.
Domain API services	`frontend/src/services/`	Organize calls for auth, customers, discovery, measurements, orders, admin, health, appointments, and international data.
Language and theme	`frontend/src/i18n/`, `frontend/src/hooks/useTheme.tsx`	English/Telugu/Hindi translation with English fallback and persistent preference; persistent dark/light theme.

3. Backend Structure

Layer	Location	Responsibilities
Application entry point	`backend/app/main.py`	Configure FastAPI, startup/shutdown lifecycle, database initialization/seeding, CORS, static files, and API router registration.
Router composition	`backend/app/api/v1/api.py`	Mount domain endpoint routers under `/api`. The same router is also mounted at the root for backward compatibility.
Endpoint handlers	`backend/app/api/v1/endpoints/`	Parse requests, declare response schemas, apply dependencies, and delegate to domain services.
Dependencies	`backend/app/api/deps.py`	Provide database sessions and current-user/role authorization dependencies.
Services	`backend/app/services/`	Implement domain rules and coordinate persistence.
Repositories	`backend/app/repositories/`	Encapsulate selected database access patterns, including user, customer, measurement, and order operations.
SQLAlchemy models	`backend/app/models/`	Define persisted entities and their relationships.
Schemas	`backend/app/schemas/`	Validate API input and serialize API responses using Pydantic.

Layer	Location	Responsibilities
Database setup	`backend/app/database/`	Create SQLAlchemy engine/session; prefer configured PostgreSQL and fall back to a local SQLite database if PostgreSQL is unreachable.

4. API Groups

The primary API prefix is `/api`. Exact operations and schemas are defined by the endpoint modules.

Prefix	Domain / typical operations
`/api/health`	Basic and detailed service/database health.
`/api/auth`	Register, login, current-user/session operations, profile changes, and logout.
`/api/tailors`	Public tailor discovery and nearby/search operations.
`/api/tailors/customers`	Tailor-owned customer directory and customer notes/profile operations.
`/api/measurement-templates`	Read available garment measurement templates and fields.
`/api/customers/measurements`	Create, read, update, and delete customer measurement profiles.
`/api/designs`	Browse and manage tailor design catalog data.
`/api/orders`	Customer order creation/list/detail plus delivery and order-related operations.
`/api/tailors/orders`	Tailor order queues and production-stage updates.
`/api/admin`	Administrator dashboard and platform/shop/customer/order/review management.
`/api/payments`	Public payment configuration, payment intent, confirmation, and order payment history.
`/api/reviews`	Create/update/delete customer reviews and read public shop review summaries.
`/api/international`	Country/currency catalogs and international validation/conversion helpers.

5. Data Model

Entity / table	Purpose and important links
`users`	Identity, credentials, active status, and role (`CUSTOMER`, `TAILOR`, or `ADMIN`).
`customers`	Customer profile linked to a user; includes country/currency and address-related data.
`customer_notes`	Notes associated with a customer and the tailor/user who created them.
`tailor_shops`	Tailor storefront linked to its owner user; shop has related location, business hours, services, and designs.
`design_categories`, `designs`, `design_images`	Categorized designs owned by a shop, with optional images.
`measurement_templates`, `measurement_fields`	Reusable garment-specific measurement definitions and their fields.
`customer_measurements`, `customer_measurement_values`	Customer measurement profile linked to a template, with values for its fields; may be associated with a shop.
`delivery_addresses`	Customer delivery details referenced by an order.
`orders`	Commission placed by a customer for a tailor shop; includes currency/payment status and links to address and order children.
`order_items`	Garment/design lines for an order, optionally referencing a saved measurement profile.

Entity / table	Purpose and important links
`order_reference_images`	Images attached to an order.
`order_status_history`	Order status changes with optional acting user.
`order_deliveries`	Delivery/tracking record associated with an order.
`payments`	Payment attempts/records associated with an order.
`shop_reviews`	Customer review for a shop and completed order; service rules restrict reviews to eligible orders and enforce one review per order.

Key cardinalities: one tailor user owns one shop; a shop can have many designs; a measurement template has many fields; a customer measurement has many values; an order has multiple items and can have status history, delivery, reference images, and payment records. Reviews reference the customer, shop, and order.

6. Request Flows

6.1 Login and protected request

1. The frontend sends credentials to the auth service endpoint.
2. On success, the frontend stores the bearer token and user data in browser storage.
3. On application startup, the auth provider checks the stored token by requesting the current user.
4. A protected frontend route checks authentication and the user role.
5. For protected API requests, the shared API client adds `Authorization: Bearer <token>`.
6. FastAPI decodes the JWT, loads the active user, and optional role dependencies authorize the endpoint. Invalid/missing sessions return 401; insufficient role returns 403.

6.2 Customer order

1. The customer selects a shop/design and measurement profile in the UI.
2. The frontend submits an order request with items, customization details, currency, and delivery address.
3. The endpoint validates the request schema and delegates to the order service.
4. The service checks related data and creates the order and child records using the request database session.
5. The API returns the order representation; customer and tailor order endpoints expose the order from their respective roles.
6. Tailor updates create/advance production status; delivery endpoints maintain delivery and tracking details.

6.3 Payment (current behavior)

1. An authenticated caller requests a payment intent for an order.
2. The payment service loads the order and calculates the amount from stored order data, rather than accepting a client-supplied total.
3. The service stores a pending payment record with the configured provider identifier.
4. A confirmation request marks the payment completed or failed and synchronizes the order payment status.

The configured provider is `MOCK_SANDBOX`; the current confirmation flow includes simulated success/failure. A live provider SDK, verified gateway callback, and webhook processing are not established by the current implementation.

7. Authorization and Validation

- Protected endpoints obtain the current user through the JWT bearer dependency.
- `RoleChecker` enforces customer, tailor, or admin roles on role-specific operations.
- Services additionally perform ownership checks for sensitive records such as customer orders, payments, reviews, and tailor-shop data.

- Pydantic schemas validate request and response shapes; international utilities validate supported country, currency, phone, and postal formats.
- Password/token secrets and production database credentials must be supplied through production environment configuration; repository defaults are development/demo values and must not be used for a live deployment.

8. Startup, Persistence, and Tests

At startup, the backend creates tables from SQLAlchemy metadata, attempts selected column synchronization, and seeds demo users, sample shops, measurement templates, and designs. A local SQLite fallback exists for development when PostgreSQL is unavailable. Alembic configuration is present, but the production migration workflow needs to be explicitly established and tested.

Repository-level end-to-end scripts cover authentication/authorization, admin, delivery, language catalogs, theme switching, and navigation. They are scripts rather than a single configured test command; a clean-environment automated test pipeline remains a release task.